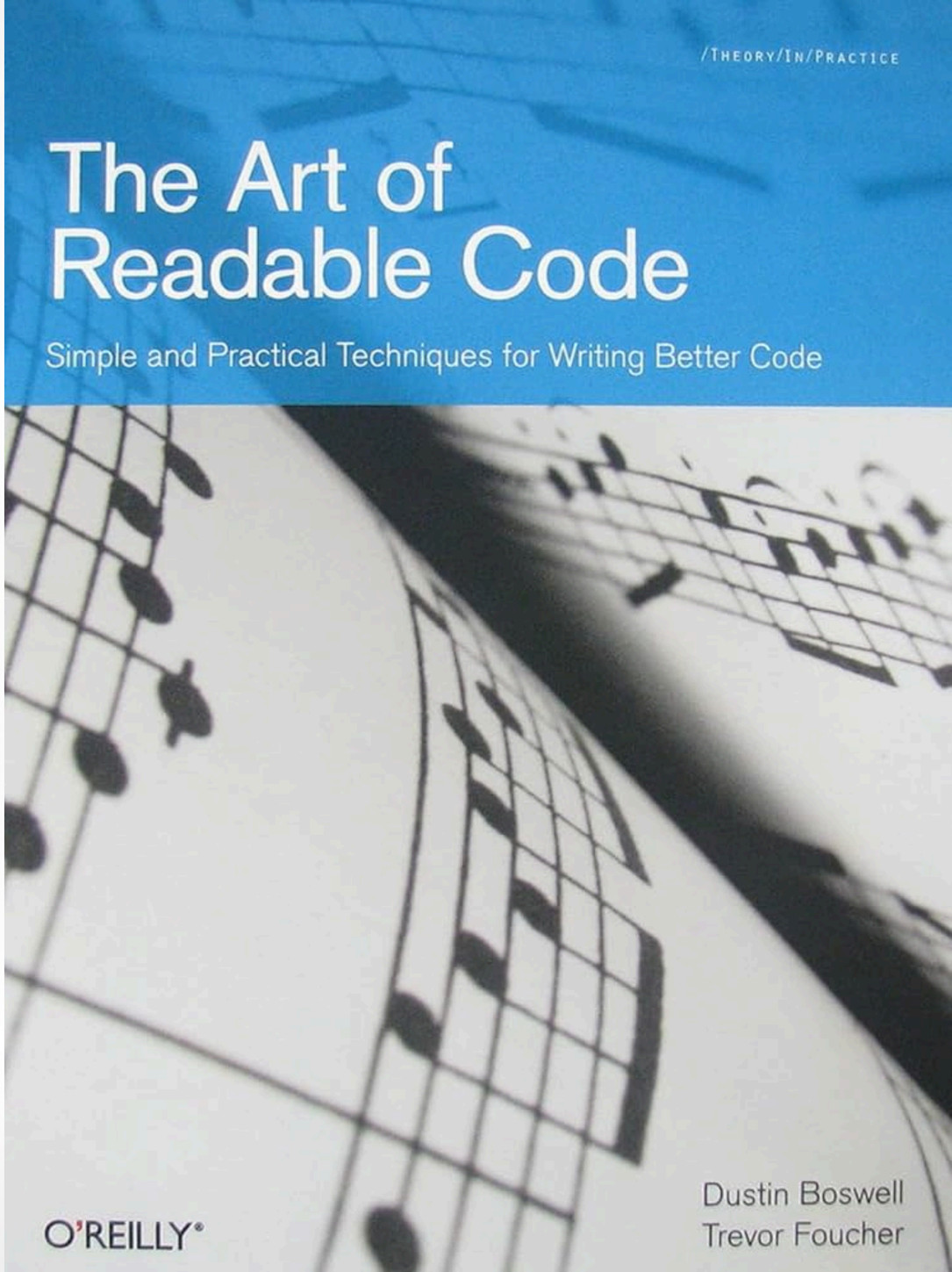
PROGRAMMING FOR DATA SCIENCE · LESSON 1

THE ART OF READABLE CODE

SUMMARY OF CHAPTER ONE

Code Should Be Easy to Understand

Ngen Tina (e20221516),
Neang Vanna (e20221650),
Yong Lyhor (e20221004)



KEY IDEA

Code should be easy to understand.

Over five years the authors collected hundreds of “bad code” examples and studied what made them bad. Every principle they found comes from this single theme.

What makes code “better”?

Most programmers decide by gut feel and intuition. Both loops below behave exactly the same, yet we know which one we prefer.

Better

```
for (Node* node = list->head; node != NULL; node = node->next)
    Print(node->data);
```

Worse

```
Node* node = list->head;
if (node == NULL) return;

while (node->next != NULL) {
    Print(node->data);
    node = node->next;
}
if (node != NULL) Print(node->data);
```



A tougher choice

Is the compact version better or worse than the longer one?

$$\text{value} = \text{mantissa} \times 2^{\text{exponent}}$$

Version 1
Compact

```
return exponent >= 0 ? mantissa * (1 << exponent) : mantissa / (1 << -exponent);
```

Version 2
Less intimidating

```
if (exponent >= 0) {
    return mantissa * (1 << exponent);
} else {
    return mantissa / (1 << -exponent);
}
```

Which criterion matters more? How do you decide how to code something?

THE FUNDAMENTAL THEOREM OF READABILITY

Code should be written to minimize the time it would take for someone else to understand it.

The metric is “time-till-understanding”: how long a typical colleague needs to read your code and understand it.

What does “understand” mean?

A reader fully understands code when they can do three things.



Make changes

Edit the code with confidence.



Spot bugs

Notice mistakes while reading.



See the big picture

Know how it interacts with the rest of the code.

Who is the “someone else”?

Even on a one-person project it matters: it may be you six months later, a teammate who joins, or “throwaway code” that gets reused.

Is smaller always better?

Less code usually takes less time to understand, but fewer lines is not the real goal.

One line
Slower to read

```
assert((!(bucket = FindBucket(key))) || !bucket->IsOccupied());
```

Two lines
Faster to read

```
bucket = FindBucket(key);
if (bucket != NULL) assert(!bucket->IsOccupied());
```

Adds a comment
Faster to read

```
// Fast version of "hash = (65599 * hash) + c"
hash = (hash << 6) + (hash << 16) - hash + c;
```

Minimizing time-till-understanding is an even better goal than fewer lines.

Does readability conflict with other goals?

The authors found that these goals barely interfere with readability.

Efficient

Even highly optimized code can be made highly readable.

Well architected

Easy-to-understand code often turns out well architected.

Easy to test

Readable code is often easy to test as well.

When in doubt, the Fundamental Theorem of Readability trumps any other rule.

No need to chase perfect factoring: ask “Is this code easy to understand?” If yes, move on.

THE HARD PART

It takes extra work

You must keep asking whether an imaginary outsider would find your code easy to understand. That means turning on a part of your brain that may not have been on while you were coding before.

The payoff

- You become a better coder
- You have fewer bugs
- You take more pride in your work
- Everyone around you loves your code

Key takeaways

1. Code should be easy to understand: the guiding principle behind every coding choice.
2. Minimize the time it takes someone else to understand your code, even if that someone is you in six months.
3. Fewer lines often help, but clarity beats brevity.
4. Readability rarely conflicts with efficiency, architecture or testing. When in doubt, choose readable.